Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего образования
«Московский государственный технический университет имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ ИУ «ИНФОРМАТИКА И СИСТЕМЫ УПРАВЛЕНИЯ»
КАФЕДРА ИУ7 «ПРОГРАММНОЕ ОБЕСПЕЧЕНИЕ ЭВМ И ИНФОРМАЦИОННЫЕ ТЕХНОЛОГИИ»

КУРСОВАЯ РАБОТА

НА ТЕМУ:

Разработка базы данных для онлайн-версии колодостроительной настольной игры «Эпичные схватки боевых магов: Крутагидон. Дополнение Магангстеры».

Студент	ИУ7-64Б		Серышева Д.С.
	(группа)	(подпись, дата)	(И.О. Фамилия)
Руководитель курсовой работы			Кострицкий А.С.
		(подпись, дата)	(И.О. Фамилия)
Консультант			
		(подпись, дата)	(И.О. Фамилия)

2026 г.

РЕФЕРАТ

Курсовая работа 47 с., 8 рис., 13 ист., 2 табл., 2 прил.

Ключевые слова: СУБД, БД, Postgresql, minIO, Python

Цель работы: Результаты работы:

— проведён анализ предметной области;

— формализованы данные и роли пользователей;

— спроектирована и разработана база данных, описаны её сущности и связи;

— спроектирован и разработан триггер валидации карты;

— спроектирована и разработана хранимая процедура проверки покупки карты;

— реализован сайт, использующий базу данных;

— проведено исследование зависимости времени формирования персональной колоды от количества карт игрока и структуры данных их хранения.

Исследование показало, что производительность операций формирования руки игрока зависит как от размера колоды, так и от используемой структуры данных. При больших размерах колод структура `deque` обеспечивает более высокую производительность, а наиболее затратной операцией является перемешивание колоды.

Содержание

РЕФЕРАТ **4**

ВВЕДЕНИЕ **7**

1 Аналитическая часть **8**
1.1 Описание предметной области 8
1.2 Типы карт 9
1.3 Механика работы рынка карт 10
1.4 Категории пользователей 10
1.5 Требования к разрабатываемому приложению 12
1.6 Формализация данных 12
1.7 Выбор модели хранения данных 15

2 Конструкторский раздел **16**
2.1 Схема базы данных 16
2.2 Ограничения целостности 16
2.3 Нормальная форма 17
2.3.1 Таблица пользователей 17
2.3.2 Таблица ролей 18
2.3.3 Таблица игр 18
2.3.4 Таблица игроков 18
2.3.5 Таблица карт 19
2.3.6 Таблица типов карт 19
2.3.7 Таблица колод 20
2.3.8 Таблица карт в колодах 20
2.3.9 Таблица изображений 20
2.4 Проектирование хранимой процедуры покупки карты 21
2.5 Проектирование триггера 23
2.6 Ролевая модель 24

3 Технологический раздел **26**
3.1 Средства реализации 26
3.2 Создание базы данных 26
3.3 Роли и их права 27
3.4 Триггер валидации карты 27
3.5 Процедура проверки покупки 27

3.6 Redis (кэширование) . . . 27
3.7 Пользовательский интерфейс . . . 28
3.8 Тестирование . . . 28
3.8.1 Тестирование приложения . . . 28
3.8.2 Тестирование триггера валидации карты . . . 29
3.8.3 Тестирование процедуры проверки покупки . . . 30

4 Исследовательский раздел . . . 31
4.1 Цель исследования . . . 31
4.2 Методика проведения исследования . . . 31
4.3 Результаты исследования . . . 32

ЗАКЛЮЧЕНИЕ . . . 34

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ . . . 35

Приложение А . . . 37

Приложение Б . . . 48

ВВЕДЕНИЕ

Современные многопользовательские онлайн-приложения предъявляют высокие требования как к структуре данных, так и к надёжности бизнес-логики и доступности сервиса. В играх с коллекционированием карт важны производительность операций с колодами, правильность валидации данных карт и надёжный контроль прав доступа для разных ролей пользователей.

Целью данной работы является разработка базы данных для прототипа игры «Эпичные схватки боевых магов: Крутагидон».

Для достижения поставленной цели необходимо решить следующие задачи:

1) проанализировать предметную область;
2) формализовать модель данных и выделить роли пользователей;
3) спроектировать базу данных, описать её сущности и связи;
4) выбрать систему управления базами данных;
5) реализовать базу данных и сайт с игрой;
6) провести исследование производительности операций с колодами.

1 Аналитическая часть

В аналитической части проведён анализ предметной области онлайн-версии настольной игры «Эпичные схватки боевых магов: Крутагидон», в том числе описание типов карт, механики работы рынка и условий победы. Сформулированы требования и ограничения к приложению, формализованы данные и роли пользователей.

1.1 Описание предметной области

Онлайн-версия упомянутой колодостроительной (deck-building) игры представляет собой многопользовательскую карточную игру, в которой игроки сражаются между собой, используя карты из общей системы карт. Для реализации используются механики, описанные в правилах игры, но некоторые объекты переименованы [1].

Игровой процесс строится на последовательности ходов *игроков* — участников игровой сессии, обладающих параметрами состояния (например, здоровье, очки крутости) и взаимодействующих с игровыми объектами в процессе игры. Во время своего хода игрок может разыгрывать карты из своей руки, получать игровой ресурс — *эхо* — и использовать его для покупки новых карт из рынка. Купленные карты добавляются в личную колоду игрока и могут быть использованы в дальнейшем. В конце каждого хода все карты, с которыми игрок начинал свой ход, сбрасываются, а на руку набираются новые [1].

Игрок обладает личной колодой карт, которая используется в процессе игры. Колода формируется из стартовых карт (началок), а затем пополняется новыми картами, приобретёнными игроком на рынке. Механика работы рынка приведена ниже. В ходе игры карты могут находиться в различных состояниях (колодах): рука, добор, стол, сброс, рынок, основная колода и игровой сброс [1].

Типы колод:

— рука (hand) — просматриваемые и разыгрываемые карты, с которыми игрок начинает ход;

— колода добора (draw) — колода игрока, из которой формируется рука и осуществляется добор карт;

— стол (table) — карты, которые игрок разыграл на текущем ходу, доступны к просмотру всем игрокам;

— сброс (discard) — колода для сброшенных карт, формирует колоду добора, когда в ней заканчиваются карты;

— рынок (market) — колода из 5 открытых карт, которые доступны для просмотра всем игроками и покупки игроку в свой ход;

— основная игровая колода (deck) — колода, из которой формируется рынок;

— сброс игры (banish) — карты, которые сбрасываются без возврата в игру.

В течение ходов игроки могут получать урон от атак других игроков или атаковать

самостоятельно, пополнять руку новыми картами, лечиться, набирать внутриигровую валюту покупки карт — эхо. Атаки уменьшают количество очков здоровья игрока и, если здоровье игрока становится меньше или равно 0, то игрок на этом ходу умирает, получает «памятку о бренности бытия», отмечающую его смерть, и сразу же восстанавливается со стандартным количеством очков здоровья [1].

Игра заканчивается, когда все выделенные на игру памятки о бренности бытия заканчиваются. Победитель определяется подсчётом очков крутости, которые складываются из очков крутости каждой карты, которой обладает игрок на момент завершения игры [1].

Для корректной работы системы необходимо хранить информацию об игроках, текущих партиях, составе колод игроков и колод игры, картах и типах карт.

1.2 Типы карт

Карты являются основным элементом для реализации игровой механики. Каждая карта обладает определённым набором характеристик и эффектов и принадлежит определенному типу.

В системе используются следующие типы карт: началка, существа (дварф, эльф, дракон, волшебник), штуки и чертоги.

Началка — карты стартовой колоды игроков.

Дварф, эльф, дракон, и волшебник — карты существ с определёнными характеристиками.

Штука — карты предметов с определёнными характеристиками.

Чертог — карты локаций, дающие постоянные эффекты.

Карты могут обладать следующими эффектами: защита, добор, атака, хилл, буст руки и буст эхо.

Защита — позволяет избежать атаки другого игрока.

Добор — позволяет взять и разыграть дополнительные карты.

Атака — позволяет атаковать другого игрока.

Хилка — позволяет восстановить часть здоровья.

Буст руки — позволяет разыгрывать больше карт, действует постоянно.

Буст эхо — увеличивает базовое значение ресурса эхо игрока, действует постоянно.

Каждая карта имеет следующие свойства:

— название;

— тип карты;

— изображение;

— эффект карты и его описание;

— числовая характеристика эффекта;

— значение эхо;

— стоимость для покупки;

— количество очков крутости.

1.3 Механика работы рынка карт

В каждой партии формируется рынок карт, из которого игроки могут приобретать новые карты. Рынок состоит из пяти открытых карт, доступных для покупки, и закрытой колоды, из которой рынок обновляется в начале каждого хода [1].

После розыгрыша своих карт игрок может купить любое количество открытых карт рынка, на которое у него хватает накопленного эхо. После покупки карта удаляется из рынка и добавляется в личную колоду игрока [1].

На покупку карт накладываются следующие ограничения:

— нельзя купить карт больше, чем есть открытых карт на рынке;

— одну и ту же карту нельзя купить больше одного раза;

— нельзя купить карту, отсутствующую на открытом рынке.

Для покупки карт используется накапливаемое игроком в течение хода эхо. Каждая разыгранная карта может как увеличить, так и уменьшить или не изменить текущее значение эхо [1].

Значение эхо хранится для каждого игрока отдельно, накапливается в течение хода и после него сбрасывается до нуля (или базового значения конкретного игрока, если он использовал карты, меняющие этот показатель) [1].

При покупке карт с рынка текущее значение эхо уменьшается на стоимость купленной карты. Карта может быть куплена только в том случае, когда текущее значение эхо игрока не меньше стоимости карты [1].

1.4 Категории пользователей

В системе предусмотрено несколько категорий пользователей, обладающих разными правами доступа:

Неавторизованный пользователь (гость) — пользователь, который может просматривать правила игры и каталог карт.

Авторизованный пользователь — пользователь, который может делать то же самое, что и неавторизованный, а также может создавать игры или присоединяться к уже созданным.

Игрок — пользователь, находящийся в игровой партии, способный выполнять ходы.

Модератор — пользователь, обладающий всеми возможностями авторизованного пользователя, который может блокировать и разблокировать других пользователей, просматривать текущие игры, временно их останавливать или завершать.

Мастер — пользователь, который может делать всё то же, что и модератор, а также добавлять новые карты, менять или удалять уже существующие.

Диаграмма вариантов использования разрабатываемой системы представлена на рисунке 1.1.

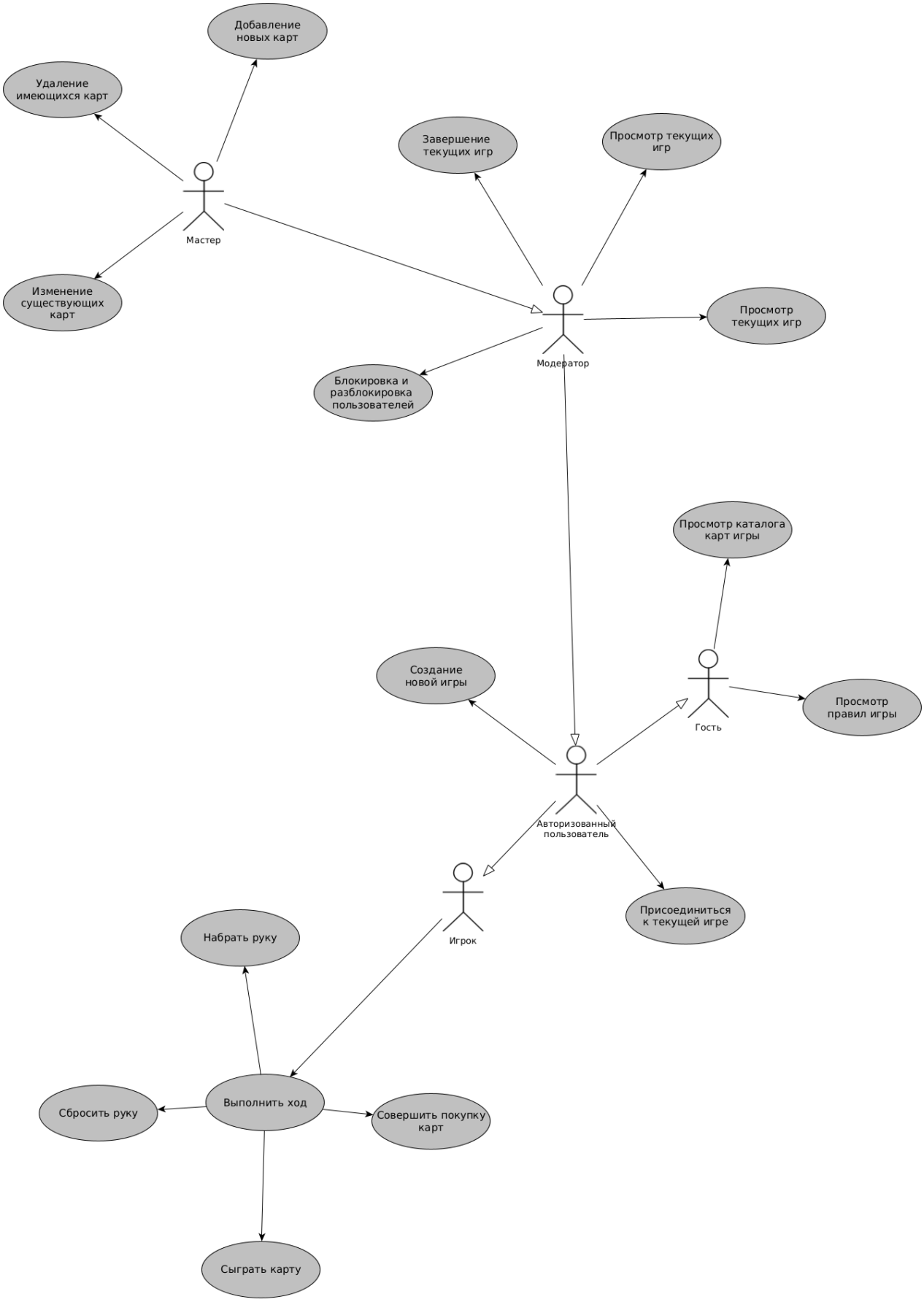


Рисунок 1.1 — Диаграмма вариантов использования разрабатываемой системы

1.5 Требования к разрабатываемому приложению

Разрабатываемый сайт предназначен для одного или нескольких гостей, способных просматривать игровые карты и правила игры, и пользователей, способных авторизоваться и быть участниками игровых партий.

Таким образом, разрабатываемое приложение должно обладать следующей функциональностью:

1) управление игровыми сессиями (создание, подключение и завершение игры);

2) взаимодействие игроков в рамках игровой сессии;

3) отображение текущего состояния игры (поле, карты, колоды и прочие игровые сущности);

4) выполнение игровых действий пользователем (применение эффектов карт, покупка новых);

5) обработка логики игры (подсчёт очков, применение правил, переходы между состояниями);

6) поддержка работы с колодами карт (формирование, перемешивание, добор и сброс карт);

Таким образом, приложение должно обеспечивать полный цикл управления игровыми сессиями, включая создание, участие в игре и просмотр результатов.

1.6 Формализация данных

В таблице 1.1 представлены сущности, используемые в базе данных.

Таблица 1.1 — Сущности и их свойства

Сущность	**Свойство**	**Метка**	**Тип**	**Атрибут БД**
Пользователь	Псевдоним	Имя пользователя, уникально	Строка	`username`
	Эл. почта	Адрес почты, уникально	Строка	`e-mail`
	Пароль	Хеш пароля	Строка	`password`
	Дата рег.	Время создания аккаунта	DateTime	`register_date`
Игра	Текущий ход	Номер текущего хода	Целое число	`current_move`
	Памятки	Оставшееся кол-во памяток	Целое число	`memos`
	Время начала	Момент старта партии	DateTime	`time_start`

Таблица 1.1 — Сущности и их свойства

Сущность	Свойство	Метка	Тип	Атрибут БД
	Победитель	Внешний ключ к Players, может быть NULL	Целое число	`winner`
Игрок	Пользователь	Внешний ключ к Users	Целое число	`user_id`
	Игра	Внешний ключ к Games	Целое число	`game_id`
	Здоровье	Текущее здоровье	Целое число	`health`
	Лимит руки	Макс. карт в руке	Целое число	`hand_limit`
	Базовое эхо	Базовый ресурс	Целое число	`base_echo`
	Порядок хода	Очередность хода игрока	Целое число	`turn`
Карта	Название	Название карты	Строка	`title`
	Эхо	Количество эхо карты	Целое число	`echo`
	Стоимость	Цена покупки	Целое число	`cost`
	Сила	Значение эффекта	Целое число	`power`
	Тип карты	Внешний ключ к Types	Целое число	`card_type`
	Очки крутости	Очки за карту	Целое число	`cool_points`
Тип карты	Действие	Тип эффекта	Строка	`action`
	Постоянность	Признак постоянного эффекта	Логический	`if_permanent`
Колода	Тип	Тип (market, hand и т.д.)	Строка	`type`
	Открытость	Могут карты быть просмотрены или нет	Логический	`if_open`
	Владелец	Внешний ключ к Players, может быть NULL	Целое число	`owner_id`
Карта в колоде	Колода	Внешний ключ к Decks	Целое число	`deck_id`

Таблица 1.1 — Сущности и их свойства

Сущность	**Свойство**	**Метка**	**Тип**	**Атрибут БД**
	Карта	Внешний ключ к Cards	Целое число	`card_id`
	Позиция	Порядковый номер карты в стопке	Целое число	`position`
Изображение	Название	Имя файла/заголовка	Строка	`title`
	Файл	Путь к файлу изображения	Path	`file`

На рисунке 1.2 представлена диаграмма сущность-связь, отражающая структуру базы данных и связи между сущностями.

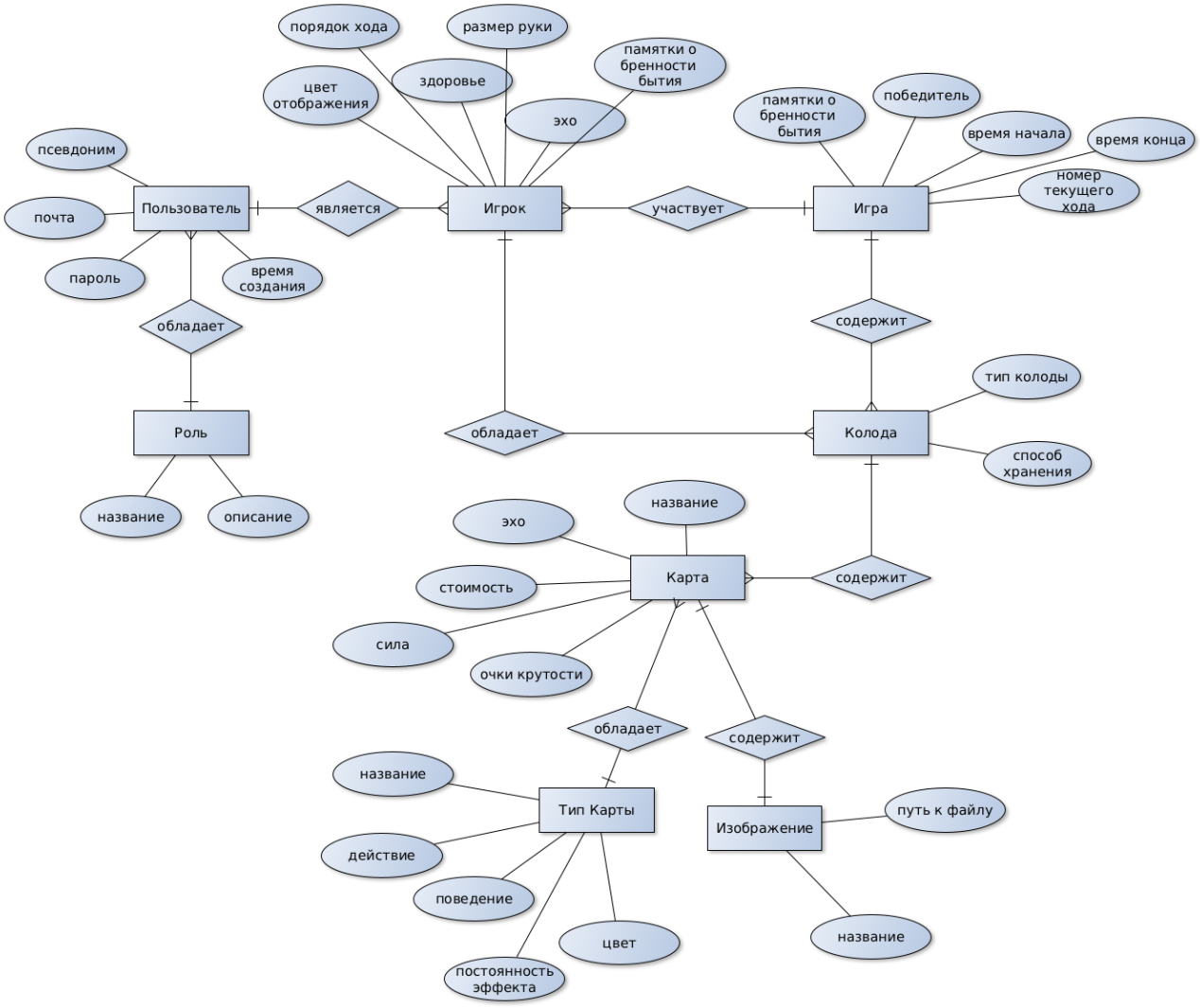


Рисунок 1.2 — Диаграмма сущность-связь в нотации Чена

1.7 Выбор модели хранения данных

В разрабатываемой системе предполагается хранение структурированных данных предметной области, включающих сущности (например, пользователи, игровые объекты, карты) и связи между ними. Для каждой сущности задан фиксированный набор атрибутов и предусмотрены взаимосвязи, требующие поддержания целостности и согласованности данных [2].

Существует несколько основных моделей хранения данных:

Дореляционная модель — данные хранятся в виде иерархических структур, файлов или сетевых баз данных. Эта модель не предполагает жёсткой схемы таблиц и характеризуется сложностью в обеспечении ссылочной целостности. Операции с данными требуют специализированного программного обеспечения, а структура данных тесно связана с применением. Дореляционный подход хорошо подходит для специализированных приложений, но неудобен для систем, требующих гибкости и универсальности [2].

Реляционная модель — информация представляется в виде таблиц с явно определёнными атрибутами, первичными и внешними ключами, ограничениями целостности и уникальности. Реляционная модель обеспечивает строгое разделение структуры данных от логики приложения, позволяет использовать стандартный язык запросов (SQL), удобна для сложных операций и обеспечивает надёжность данных через механизмы транзакций [3].

Постреляционная модель — Такие БД основаны на реляционных, однако отходят от их принципов в пользу более эффективного решения специфических задач. Одним из примеров таких особенностей выступает возможность вложенного хранения таблиц (хранения таблиц в таблицах), что невозможно в реляционных БД [4].

С учётом требований разрабатываемой системы наиболее подходящей является реляционная модель данных. Она позволяет явно определить структуру данных (пользователи, игры, игроки, карты, колоды), обеспечить строгое соблюдение ограничений целостности, поддержать сложные многотабличные запросы для анализа состояния игры и гарантировать консистентность данных при одновременном доступе нескольких игроков. Реляционная модель удобна для выполнения сложных запросов и фильтрации информации.

Вывод

В аналитической части была рассмотрена предметная область, были рассмотрены и формализованы данные, подлежащие хранению и выбрана реляционная модель хранения данных.

2 Конструкторский раздел

В конструкторской части были разработаны реляционная схема базы данных и ролевая модель, рассмотрены ограничения целостности и разработана процедура

2.1 Схема базы данных

В аналитической части была выбрана реляционная модель данных, поэтому данные организованы в виде связных таблиц. На рисунке 2.1 представлена реляционная схема базы данных, основанная на диаграмме сущность-связь 1.2, отражающей структуру базы данных и связи между сущностями.

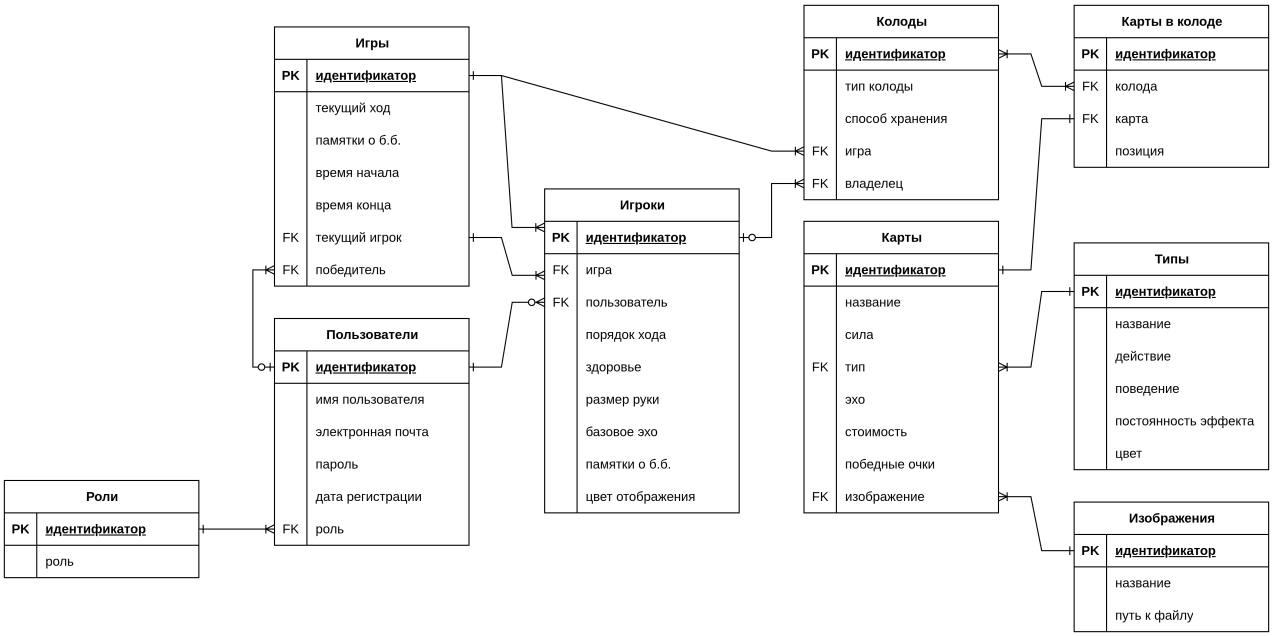


Рисунок 2.1 — Реляционная схема базы данных

2.2 Ограничения целостности

В системе введены следующие ограничения целостности данных:

— время начала партии должно быть раньше времени окончания партии, если партия завершена;

— победитель партии может быть указан только для завершённой партии;

— размер руки, значения здоровья, текущего эхо, базового эхо и количества памяток игрока не могут быть отрицательными;

— стоимость карты, значение эхо, количество очков крутости и сила эффекта карты не могут быть отрицательными;

— тип колоды должен принимать одно из допустимых значений: колода добора, рука, стол, сброс игрока, общий игровой сброс, открытый рынок или закрытый рынок;

— в рамках одной партии у одного игрока не должно быть более одной колоды каждого личного типа: колоды добора, руки, стола и сброса;

— в рамках одной партии не должно быть более одной открытой колоды рынка, одной закрытой колоды рынка и одной колоды игрового сброса;

— номера позиций карт должны быть уникальными в рамках одной колоды и иметь неотрицательные значения;

— купить можно только карту, находящуюся в открытом рынке текущей партии;

— покупка карты возможна только при достаточном количестве эхо у игрока;

— после покупки карта должна быть удалена из открытого рынка и добавлена в сброс игрока;

— если количество оставшихся памяток о бренности бытия становится равным нулю, партия должна перейти в завершённое состояние.

2.3 Нормальная форма

Для проверки корректности проектирования базы данных была выполнена проверка отношений на соответствие третьей нормальной форме (3НФ) [3].

Отношение находится в третьей нормальной форме, если:

— оно находится во второй нормальной форме;

— каждый неключевой атрибут нетранзитивно зависит от первичного ключа.

2.3.1 Таблица пользователей

Отношение:

$$Users(id, username, email, password, register_date, role)$$

Функциональные зависимости:

$$id \to username, email, password, register_date, role$$

Также:

$$username \to id$$

$$email \to id$$

Первичный ключ:

$$id$$

Кандидатные ключи:

$$username,\ email$$

Все неключевые атрибуты полностью функционально зависят от первичного ключа.

Транзитивные зависимости отсутствуют.

Следовательно, отношение `Users` находится в 3НФ.

2.3.2 Таблица ролей

Отношение:

$$Roles(id, role)$$

Функциональные зависимости:

$$id \to role$$

Первичный ключ:

$$id$$

Транзитивные зависимости отсутствуют.

Следовательно, отношение `Roles` находится в 3НФ.

2.3.3 Таблица игр

Отношение:

$$Games(id, current_move, memos, time_start, time_end, winner)$$

Функциональные зависимости:

$$id \to current_move, memos, time_start, time_end, winner$$

Первичный ключ:

$$id$$

Все атрибуты зависят только от первичного ключа. Транзитивные зависимости отсутствуют.

Следовательно, отношение `Games` находится в 3НФ.

2.3.4 Таблица игроков

Отношение:

$$Players(id, game_id, user_id, color, health, hand_limit, base_echo, memos, turn)$$

Функциональные зависимости:

$$id \to game_id, user_id, color, health, hand_limit, base_echo, memos, turn$$

Первичный ключ:

$$id$$

Все неключевые атрибуты полностью зависят от первичного ключа. Транзитивные зависимости отсутствуют.

Следовательно, отношение `Players` находится в 3НФ.

2.3.5 Таблица карт

Отношение:

$$Cards(id, title, creature, echo, cost, Lpower, card_type, image_id, cool_points)$$

Функциональные зависимости:

$$id \rightarrow title, creature, echo, cost, power, card_type, image_id, cool_points$$

Первичный ключ:

$$id$$

Все атрибуты полностью функционально зависят от первичного ключа. Транзитивные зависимости отсутствуют.

Следовательно, отношение `Cards` находится в 3НФ.

2.3.6 Таблица типов карт

Отношение:

$$Types(id, action, usage_pattern, if_permanent, color)$$

Функциональные зависимости:

$$id \rightarrow action, usage_pattern, if_permanent, color$$

Первичный ключ:

$$id$$

Все атрибуты зависят только от первичного ключа. Транзитивные зависимости отсутствуют.

Следовательно, отношение `Types` находится в 3НФ.

2.3.7 Таблица колод

Отношение:

$$Decks(id, type, if_open, owner_id)$$

Функциональные зависимости:

$$id \to type, if_open, owner_id$$

Первичный ключ:

$$id$$

Все атрибуты зависят только от первичного ключа. Транзитивные зависимости отсутствуют.

Следовательно, отношение `Decks` находится в 3НФ.

2.3.8 Таблица карт в колодах

Отношение:

$$DecksCards(id, deck_id, card_id, position)$$

Функциональные зависимости:

$$id \to deck_id, card_id, position$$

Дополнительно:

$$(deck_id, position) \to card_id$$

Первичный ключ:

$$id$$

Альтернативный ключ:

$$(deck_id, position)$$

Все атрибуты полностью функционально зависят от ключей. Транзитивные зависимости отсутствуют.

Следовательно, отношение `DecksCards` находится в 3НФ.

2.3.9 Таблица изображений

Отношение:

$$Images(id, title, file)$$

Функциональные зависимости:

$$id \to title, file$$

Первичный ключ:

$$id$$

Все атрибуты полностью зависят от первичного ключа. Транзитивные зависимости отсутствуют.

Следовательно, отношение `Images` находится в 3НФ.

Во всех отношениях:

— отсутствуют повторяющиеся группы атрибутов;

— все неключевые атрибуты полностью функционально зависят от ключей;

— отсутствуют транзитивные зависимости между неключевыми атрибутами.

Следовательно, отношения базы данных находятся в третьей нормальной форме.

2.4 Проектирование хранимой процедуры покупки карты

Для выполнения операции покупки карты с открытого рынка в базе реализована хранимая процедура $buy_card_from_market$. Она используется для того, чтобы все проверки и все изменения происходили вместе.

Атомарность операции покупки карты обеспечивается механизмом транзакций СУБД. Все изменения данных выполняются в рамках одной транзакции. В случае возникновения ошибки на любом этапе выполнения процедуры формируется исключение, что приводит к откату транзакции. Таким образом, либо все изменения (списание эхо, удаление карты из рынка и добавление карты в сброс) выполняются полностью, либо не выполняется ни одно из них.

Транзакционность операции обеспечивается тем, что выполнение хранимой процедуры происходит в рамках одной транзакции СУБД. Все проверки и изменения состояния выполняются последовательно и фиксируются только при успешном завершении процедуры. В случае возникновения ошибки транзакция откатывается.

Схема хранимой процедуры представлена на рисунке 2.2.

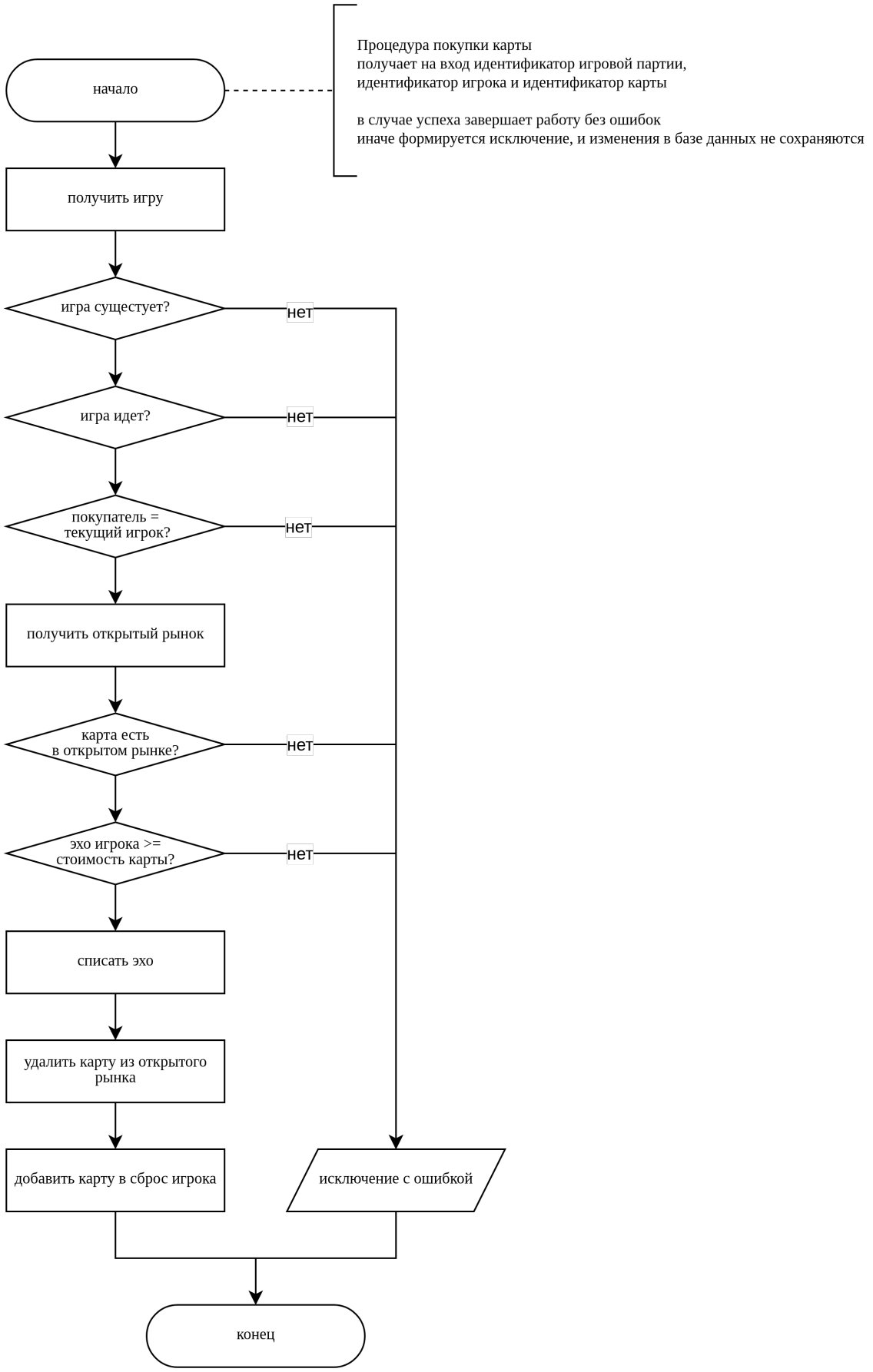


Рисунок 2.2 — Схема хранимой процедуры покупки карты из открытого рынка

2.5 Проектирование триггера

Для поддержания целостности данных необходимо, чтобы характеристики карты соответствовали установленным бизнес-правилам. Триггер срабатывает перед добавлением или изменением записи в таблице `cards` и проверяет следующие ограничения:

— стоимость карты (`cost`) должна быть положительной и не превышать 7;

— сила карты (`power`) должна быть положительной; для карт с типом `buff_*` сила не должна превышать 2;

— очки крутости (`cool_points`) должны находиться в диапазоне от -1 до 5 включительно;

— каждая карта должна иметь тип (`card_type_id` не может быть NULL);

— для карт с типом `buff_*` поле `creature` должно содержать название локации (не может быть пустым).

Если хотя бы одно из условий не выполняется, триггер прерывает операцию и откатывает изменение.

Схема работы такого триггера представлена на рисунке 2.3.



Рисунок 2.3 — Схема триггера валидации карты

2.6 Ролевая модель

На основе пользователей приложения, описанных в аналитической части, были сформированы следующие роли и права доступа к базе данных:

— пользователь (гость) — имеет доступ к каталогу игровых карт и правилам;

— авторизованный пользователь — так же может просматривать и создавать записи в таблице игровых партий;

— игрок — имеет доступ к просмотру, изменению записей таблиц игровых колод и просмотру таблицы игроков в игре;

— модератор — обладает всеми правами авторизованного пользователя и может изменять и удалять записи в таблицах пользователей и игровых партий;

— мастер — всё то же, что и модератор, а также изменение и удаление записей в каталоге карт.

Описание прав доступа к таблицам на создание, чтение, изменение и удаление данных (CRUD) представлено в таблице 2.1.

В таблице используются следующие сокращения:

— польз. — пользователь;

— автор. польз. — авторизированный пользователь.

Таблица 2.1 — Права доступа к таблицам

Таблица	**Польз.**	**Автор. польз.**	**Игрок**	**Модератор**	**Мастер**
Пользователь	-	-	-	CRUD	CRUD
Игра	-	CR	R	CRUD	CRUD
Игрок	-	CR[1]	R	CRUD	CRUD
Карта	R	R	R	R	CRUD
Тип карты	R	R	R	R	CRUD
Изображение	R	R	R	R	CRUD
Колода	-	-	R	R	R
Карта в колоде	-	-	RUD[2]	R	R

Вывод

В результате конструкторского раздела была разработана реляционная схема базы данных, рассмотрены ограничения целостности на неё, описана ролевая модель, реализованы хранимая процедура покупки карты и триггер валидации карты.

[1] Только для создания записи о собственном участии в партии

[2] Изменение выполняется только в рамках игровых операций или хранимых процедур

3 Технологический раздел

В рамках технологической части были выбраны средства реализации проекта, описаны программы для создания и запуска базы данных, триггера валидации карты и хранимой процедуры проверки покупки, описанных в конструкторской части. Был описан алгоритм тестирования разработанных триггера и процедуры.

3.1 Средства реализации

В качестве СУБД был выбран Postgresql [5] так как он позволяет реализовать разработанную структуру данных и ролевую модель, а также поддерживает реализацию триггеров на добавление и обновление данных.

Для хранения файлов, в частности изображений карт, было выбрано объектное хранилище Minio [6] так как оно позволяет легко и быстро развернуть хранилище объектов, независимое от файловой и операционной системы.

В качестве основного языка для написания проекта был выбран Python [7] так как средств языка достаточно для разработки приложения, есть библиотеки для взаимодействия с Postgresql [8] и Minio [9].

3.2 Создание базы данных

На основе разработанной в конструкторском разделе реляционной схемы были реализованы таблицы пользователей, партий, игроков, карт, типов карт, изображений, колод и карт в колодах. В программе создания базы данных задаются первичные и внешние ключи, ограничения уникальности, обязательность полей и связи между таблицами.

Создание базы данных происходит в два этапа. На первом этапе создаётся схема базы данных и накладываются ограничения целостности. Для этого используются миграции Alembic [10], которые формируют и последовательно применяют SQL-команды создания и изменения таблиц. Скрипт представлен в приложении А в листинге 4.1.

На втором этапе база данных заполняется начальными данными: типами карт, изображениями и набором игровых карт. Этап заполнения базы данных представлен в листинге 4.2.

Интерфейс доступа к базе данных реализован через репозитории пользователей, карт, типов карт и партий. Репозитории используют сессии SQLAlchemy, создаваемые фабрикой `SessionFactory`, поэтому прикладная логика обращается к базе данных через методы предметного уровня, а не через SQL-запросы напрямую.

3.3 Роли и их права

В конструкторской части были выделены следующие пять ролей:

— пользователь (гость);

— авторизованный пользователь;

— игрок;

— модератор;

— мастер.

Кроме стандартных прав на уровне ролей, для дополнительной защиты данных использовалась фильтрация строк PostgreSQL `postgres-rls`, которая ограничивает доступ к записям в таблицах на уровне строк.

Код для создания ролей и определения прав доступа представлен в приложении А в листингах 4.3 — 4.8.

3.4 Триггер валидации карты

Для обеспечения целостности данных был разработан триггер, который при каждом вставлении или обновлении записи в таблице `cards` проверяет соответствие характеристик карты установленным ограничениям. Если одно из условий не выполняется, триггер прерывает операцию и откатывает изменение. Код разработанного триггера представлен в приложении А, в листинге 4.9.

3.5 Процедура проверки покупки

Для проверки покупки карты была реализована хранимая процедура, которая проверяет доступность карты, текущее эхо игрока и бизнес-правила перед совершением покупки. Если объём эхо меньше стоимости карты, процедура генерирует исключение, и операция покупки не выполняется. Код хранимой процедуры представлен в приложении А, в листинге 4.10.

3.6 Redis (кэширование)

Для повышения производительности системы при работе с каталогом карт используется Redis [14], его применение позволяет сократить количество обращений к базе данных.

В кэше сохраняются готовые JSON-ответы для эндпоинтов, предназначенных только для просмотра данных каталога карт. При поступлении запроса система сначала проверяет наличие соответствующих данных в Redis. Если данные найдены, ответ формируется непосредственно из кэша, в противном случае информация извлекается из базы данных, преобразуется в JSON-представление и сохраняется в Redis для последующих запросов.

Для обеспечения актуальности данных каждой записи в кэше назначается ограниченное время жизни (TTL, Time To Live). После истечения этого времени запись автоматически уда-

ляется из Redis и при следующем обращении формируется заново на основе данных из базы данных.

При добавлении, изменении или удалении карты записи кэша, связанные с каталогом карт, принудительно удаляются.

3.7 Пользовательский интерфейс

В качестве пользовательского интерфейса было разработано веб-приложение с игрой, предоставляющее описанную в аналитической части функциональность. Приложение разделено на основные компоненты, представляющие различные этапы взаимодействия пользователя с системой: авторизация, просмотр каталога карт, редактирование колод и процесс игровой сессии. Описание компонентов пользовательского интерфейса представлено в UML формате на рисунке 3.1.

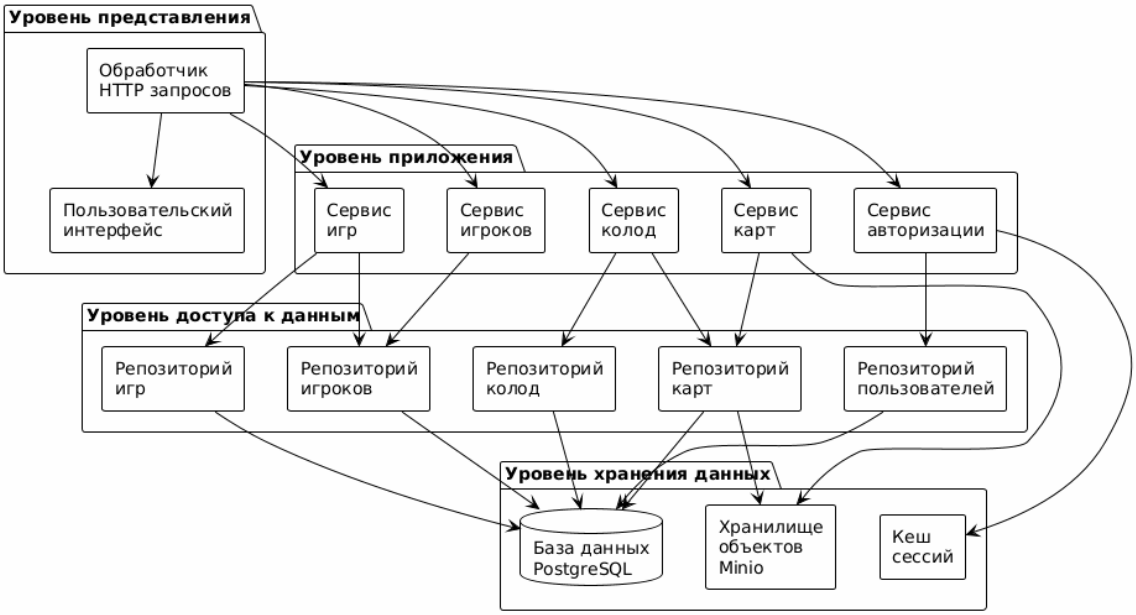


Рисунок 3.1 — Диаграмма компонентов пользовательского интерфейса

3.8 Тестирование

3.8.1 Тестирование приложения

Модульное тестирование приложения проводилось с помощью фреймворка `pytest` [**?**] и стандартной библиотеки `unittest.mock` для изоляции тестируемых компонентов.

Тестами были покрыты модули игровой логики, логики карт, управления состоянием партии, сервиса колод и контроля доступа.

Покрытие кода составило 84%.

3.8.2 Тестирование триггера валидации карты

Тестирование триггера проводилось с помощью тестов на языке Python. Были выделены следующие классы эквивалентности:

— корректная карта (все поля в допустимых пределах);

— карта с нулевой стоимостью (`cost` = 0);

— карта со стоимостью выше максимума (`cost` > 7);

— карта с нулевой силой (`power` = 0);

— карта типа `buff_*` с силой выше 2 (`power` > 2);

— карта с `cool_points` ниже минимума (< -1);

— карта с `cool_points` выше максимума (> 5);

— карта без типа (`card_type_id` = NULL);

— карта типа `buff_*` с пустым полем `creature`;

— карта типа `buff_*` с непустым полем `creature`.

Каждый класс эквивалентности был протестирован на операциях вставки и обновления. Общий алгоритм тестирования триггера имел следующий вид:

1) Создание тестовых данных в базе данных (тип карты, изображение).
2) Цикл по всем тестам:
    2.1) вставка записи карты с тестовыми данными;
    2.2) если тест позитивный:
        — проверка, что триггер не вызвал ошибку;
        — проверка, что запись успешно добавлена в таблицу;
    2.3) иначе:
        — проверка, что триггер вызвал ошибку;
        — проверка, что запись не добавлена в базу данных;
    2.4) очистка таблицы карт.
3) Цикл по всем тестам:
    3.1) создание корректной записи карты;
    3.2) обновление записи согласно тестовым данным;
    3.3) если тест позитивный:
        — проверка, что триггер не вызвал ошибку;
        — проверка, что запись успешно изменена;
    3.4) иначе:
        — проверка, что триггер вызвал ошибку;
        — проверка, что запись не изменена.

3.8.3 Тестирование процедуры проверки покупки

Тестирование хранимой процедуры проводилось с помощью тестов на языке Python. Были выделены следующие классы эквивалентности:

— корректная покупка при достаточном количестве эхо;

— покупка при недостаточном количестве эхо;

— покупка несуществующей карты;

— покупка карты, которой нет в каталоге.

Общий алгоритм тестирования процедуры имел следующий вид:

1) Создание тестовых данных: игрока с заданным количеством эхо и карты с заданной стоимостью.

2) Вызов процедуры с тестовыми параметрами.

3) Если тест позитивный:

   — проверка, что процедура не вызвала исключение;

   — проверка, что стоимость карты вычтена из баланса эхо игрока;

   — проверка, что карта добавлена в колоду игрока.

4) Иначе:

   — проверка, что процедура вызвала исключение;

   — проверка, что баланс эхо игрока не изменился;

   — проверка, что карта не добавлена в колоду игрока.

Вывод

В результате технологической части были выбраны средства реализации работы, созданы база данных, роли, триггер и хранимая процедура. Также было разработано и протестировано приложение для игры.

4 Исследовательский раздел

4.1 Цель исследования

Целью исследования является анализ влияния размера личной колоды игрока и используемой структуры хранения данных на производительность операций завершения хода и формирования новой руки игрока.

В рамках исследования сравнивались структуры данных `list` [12] и `deque` [13], используемые для хранения игровых колод. Также оценивалось влияние необходимости перемешивания сброса на общее время выполнения операций.

4.2 Методика проведения исследования

Для проведения исследования была реализована программа, моделирующая завершение хода игрока.

Во время выполнения операции выполнялись следующие шаги:

1) перенос карт из руки игрока в сброс;
2) перенос карт из сброса обратно в колоду добора;
3) перемешивание колоды;
4) добор новой руки.

Исследование проводилось для двух вариантов организации хранения колоды:

— `list`;
— `deque`.

Замеры выполнялись для колод различных размеров:

— 20 карт;
— 50 карт;
— 100 карт;
— 200 карт;
— 500 карт;
— 1000 карт;
— 2000 карт.

Для уменьшения влияния случайных факторов каждый эксперимент повторялся 1000 раз, после чего вычислялось среднее время выполнения операции.

Дополнительно исследования выполнялись в двух режимах:

— без перемешивания сброса;
— с перемешиванием сброса.

Время выполнения измерялось с использованием функции `perf_counter()` языка Python.

4.3 Результаты исследования

В ходе исследования были построены графики зависимости времени выполнения операции завершения хода от размера колоды. Графики представлены на рисунках 4.1, 4.2.

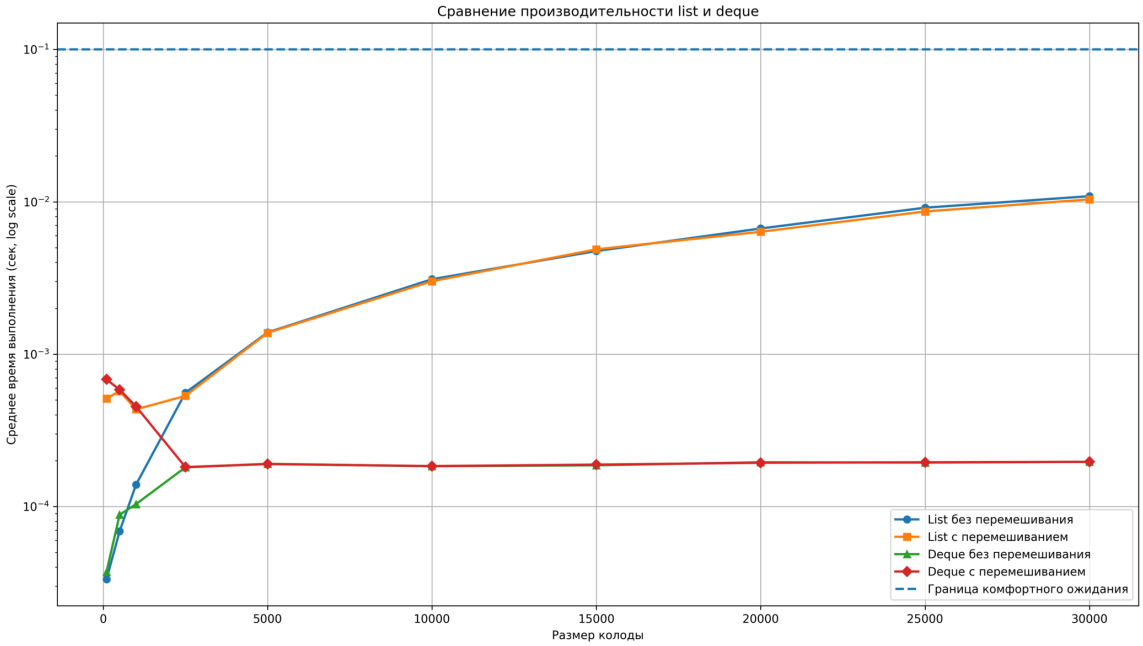


Рисунок 4.1 — График замеров времени формирования колоды с временем комфортного ожидания пользователя

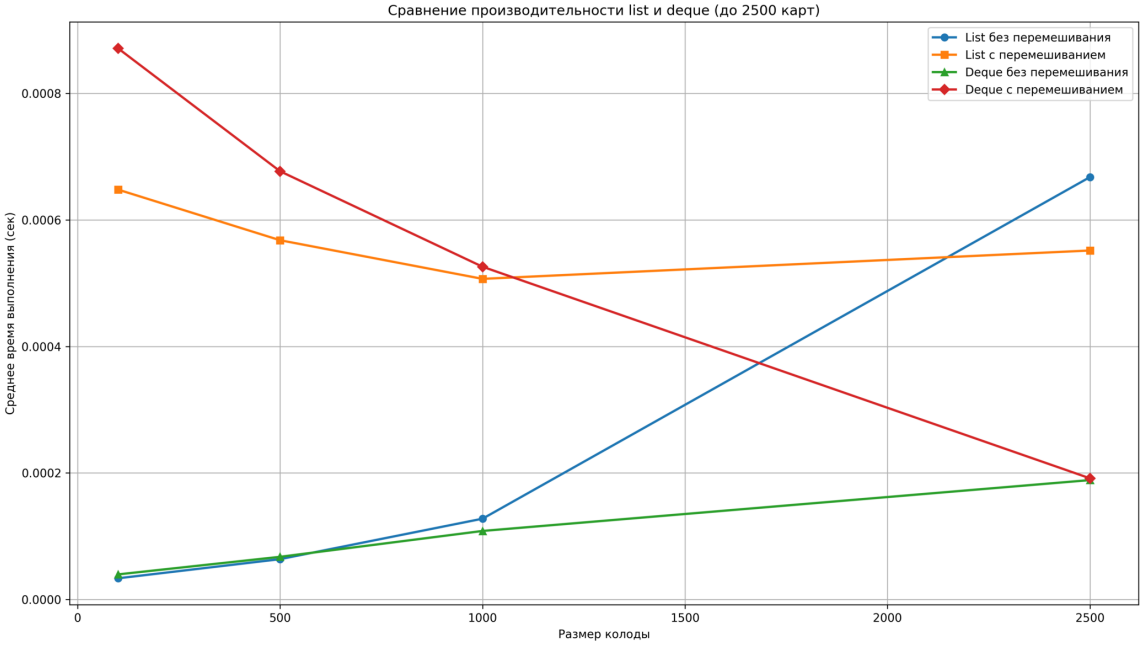


Рисунок 4.2 — График замеров времени формирования колоды на малых значениях

Результаты показали, что:

— время выполнения операций увеличивается с ростом размера колоды;

— необходимость перемешивания оказывает значительно большее влияние на производительность, чем сам размер колоды;

— структура `list` демонстрирует более высокую производительность при отсутствии перемешивания;

— при использовании перемешивания различия между `list` и `deque` уменьшаются из-за дополнительных затрат на перестановку элементов.

Также было установлено, что основное время выполнения затрачивается на операции копирования и перемешивания элементов колоды.

Вывод

В результате исследования было установлено, что производительность игровых операций зависит не только от размера колоды, но и от используемой структуры хранения данных, а также от необходимости перемешивания сброса.

Для операций последовательного хранения и добора карт структура `list` показала более высокую производительность. Однако при частых операциях удаления элементов из начала коллекции использование `deque` позволяет уменьшить накладные расходы.

Наибольшее влияние на время выполнения оказывает операция перемешивания колоды, поскольку она требует обработки всех элементов коллекции.

ЗАКЛЮЧЕНИЕ

В результате курсовой работы был проведён анализ предметной области настольной игры «Эпичные схватки боевых магов: Крутагидон». Были формализованы данные о пользователях, партиях, картах, колодах, выделены роли пользователей и их сценарии использования.

Была спроектирована реляционная база данных, для хранения данных о пользователях, играх, игроках, картах и колодах, описаны ограничения целостности, сформулированы ограничения доступа и действий каждой роли в системе.

В результате работы был разработан сайт для игры в онлайн-версию «Эпичных схваток боевых магов: Крутагидон». Разработанное ПО позволяет создавать новые игры и присоединяться к уже существующим, управлять колодами карт, просматривать информацию о картах и игроках, а также выполнять игровые действия в рамках партий.

В ходе исследования была проведена проверка корректности работы разработанной системы и основных игровых операций. Были протестированы механики покупки и розыгрыша карт, перехода хода между игроками, работы игровых колод, а также ограничения целостности базы данных, триггеры и хранимые процедуры. Результаты исследования показали корректную работу реализованной игровой логики и соответствие системы сформулированным требованиям.

В ходе работы поставленные цель и задачи были выполнены.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Мосигра. Правила игры «Эпичные схватки боевых магов. Крутагидон» // URL:

   https://www.mosigra.ru/download/rules/jepichnie-shvatki-boevih-magov-krutagidon-2
   pdf (дата обращения: 20.02.2026).

2. Конспекты лекций курса «Базы данных» за 2024 год. МГТУ им. Н.Э. Баумана, кафедра «Программное обеспечение ЭВМ и информационные технологии». Учебные материалы.

3. Дейт К. Дж. Введение в системы баз данных. — 8-е изд. — М.: Издательский дом “Вильяме”, 2005. — 1328 с.

4. Парфенов, Ю. П. Постреляционные хранилища данных : учеб. пособие. — Екатеринбург: изд-во Урал, 2016. — 120 с.

5. PostgreSQL: The World’s Most Advanced Open Source Relational Database // URL: https://www.postgresql.org/ (дата обращения: 01.05.2026).

6. MinIO: High Performance Data Store for AI & Analytics // URL: https://www.min.io/ (дата обращения: 01.05.2026).

7. Python: The Programming Language // URL: https://www.python.org/ (дата обращения: 01.05.2026).

8. Psycopg2: Python interface to PostgreSQL // URL: https://www.psycopg.org/ (дата обращения: 01.05.2026).

9. Minio Python Client Library // URL: https://docs.min.io/docs/python-client-api-reference.html (дата обращения: 01.05.2026).

10. Alembic: Database Migrations for SQLAlchemy // URL: https://alembic.sqlalchemy.org/ (дата обращения: 01.05.2026).

11. Row Level Security in PostgreSQL // URL: https://www.postgresql.org/docs/current/ddl-rowsecurity.html (дата обращения: 01.05.2026).

12. More on Lists // Python Documentation URL:

    https://docs.python.org/3/tutorial/datastructures.html#more-on-lists (дата обращения: 01.05.2026).

13. Container datatypes // Python Documentation URL:

    https://docs.python.org/3/library/collections.html#collections.deque (дата обращения: 01.05.2026).

14. Redis // Real-time data for agents & apps URL: https://redis.io/ (дата обращения: 01.05.2026).

Приложение А

Листинг 4.1 — Скрипт миграции Alembic

```
from collections.abc import Sequence
from uuid import UUID

import sqlalchemy as sa
from alembic import op
from sqlalchemy.dialects import postgresql

revision: str = "001_initial"
down_revision: str | None = None
branch_labels: str | Sequence[str] | None = None
depends_on: str | Sequence[str] | None = None

DEFAULT_CARD_TYPE_ID = UUID("6c9097de-cd28-5eab-975b-8a7de384fd6f")


def upgrade() -> None:
  op.create_table(
    "users",
    sa.Column("id", postgresql.UUID(as_uuid=True), nullable=False),
    sa.Column("username", sa.String(length=50), nullable=False),
    sa.PrimaryKeyConstraint("id"),
    sa.UniqueConstraint("username", name="uq_user_username"),
  )

  op.create_table(
    "card_types",
    sa.Column("id", postgresql.UUID(as_uuid=True), nullable=False),
    sa.Column("action", sa.String(length=50), nullable=False),
    sa.Column("usage_pattern", sa.String(length=50), nullable=False),
    sa.Column("if_permanent", sa.Boolean(), nullable=False,
       server_default="false"),
    sa.Column("color", sa.String(length=50), nullable=False,
       server_default="red"),
    sa.PrimaryKeyConstraint("id"),
  )

  op.execute(
    sa.text(
      """
```

```
        INSERT INTO card_types (id, action, usage_pattern, if_permanent,
            color)
        VALUES (:card_type_id, 'attack', 'reg', false, 'red')
        ON CONFLICT (id) DO NOTHING
        """
    ).bindparams(
        sa.bindparam(
            "card_type_id",
            value=DEFAULT_CARD_TYPE_ID,
            type_=postgresql.UUID(as_uuid=True),
        )
    )
)

op.create_table(
    "games",
    sa.Column("id", postgresql.UUID(as_uuid=True), nullable=False),
    sa.Column("host_user_id", postgresql.UUID(as_uuid=True), nullable=
        False),
    sa.Column("status", sa.String(length=50), nullable=False),
    sa.Column("cur_turn", sa.Integer(), nullable=False, server_default="
        0"),
    sa.Column("cur_player_id", postgresql.UUID(as_uuid=True), nullable=
        True),
    sa.Column("winner_id", postgresql.UUID(as_uuid=True), nullable=True)
        ,
    sa.ForeignKeyConstraint(["host_user_id"], ["users.id"]),
    sa.PrimaryKeyConstraint("id"),
)

# images table: stores image metadata for cards
op.create_table(
    "images",
    sa.Column("id", postgresql.UUID(as_uuid=True), nullable=False),
    sa.Column("title", sa.String(length=200), nullable=False),
    sa.Column("file", sa.String(length=500), nullable=True),
    sa.PrimaryKeyConstraint("id"),
)

op.create_table(
    "cards",
```

```
    sa.Column("id", postgresql.UUID(as_uuid=True), nullable=False),
    sa.Column("title", sa.String(length=200), nullable=False),
    sa.Column("creature", sa.String(length=100), nullable=False),
    sa.Column("power", sa.Integer(), nullable=False, server_default="2")
        ,
    sa.Column("echo", sa.Integer(), nullable=False, server_default="1"),
    sa.Column("cost", sa.Integer(), nullable=False, server_default="3"),
    sa.Column("cool_points", sa.Integer(), nullable=False,
        server_default="0"),
    sa.Column(
      "card_type_id",
      postgresql.UUID(as_uuid=True),
      nullable=False,
      server_default=str(DEFAULT_CARD_TYPE_ID),
    ),
    sa.Column(
      "image_id",
      postgresql.UUID(as_uuid=True),
      nullable=False,
    ),
    sa.ForeignKeyConstraint(["card_type_id"], ["card_types.id"]),
    sa.ForeignKeyConstraint(["image_id"], ["images.id"]),
    sa.PrimaryKeyConstraint("id"),
    sa.UniqueConstraint("title", name="uq_cards_title"),
  )

  op.alter_column("cards", "card_type_id", server_default=None)

  op.create_table(
    "players",
    sa.Column("id", postgresql.UUID(as_uuid=True), nullable=False),
    sa.Column("game_id", postgresql.UUID(as_uuid=True), nullable=False),
    sa.Column("user_id", postgresql.UUID(as_uuid=True), nullable=False),
    sa.Column("nickname", sa.String(length=50), nullable=False),
    sa.Column("turn_order", sa.Integer(), nullable=False, server_default
        ="0"),
    sa.Column("health", sa.Integer(), nullable=False, server_default="20
        "),
    sa.Column("base_echo", sa.Integer(), nullable=False, server_default=
        "0"),
    sa.Column("cur_echo", sa.Integer(), nullable=False, server_default="
```

```
        0"),
    sa.Column("hand_size", sa.Integer(), nullable=False, server_default=
        "5"),
    sa.ForeignKeyConstraint(["game_id"], ["games.id"], ondelete="CASCADE
        "),
    sa.ForeignKeyConstraint(["user_id"], ["users.id"]),
    sa.PrimaryKeyConstraint("id"),
    sa.UniqueConstraint("game_id", "user_id", name="uq_players_game_user
        "),
  )

  op.create_foreign_key(
    "fk_games_winner_id_players",
    "games",
    "players",
    ["winner_id"],
    ["id"],
  )

  op.create_table(
    "decks",
    sa.Column("id", postgresql.UUID(as_uuid=True), nullable=False),
    sa.Column("type", sa.String(length=50), nullable=False),
    sa.Column("game_id", postgresql.UUID(as_uuid=True), nullable=True),
    sa.Column("player_id", postgresql.UUID(as_uuid=True), nullable=True)
        ,
    sa.Column("if_open", sa.Boolean(), nullable=False, server_default="
        false"),
    sa.ForeignKeyConstraint(["game_id"], ["games.id"]),
    sa.ForeignKeyConstraint(["player_id"], ["players.id"]),
    sa.PrimaryKeyConstraint("id"),
  )

  op.create_table(
    "deck_cards",
    sa.Column("id", postgresql.UUID(as_uuid=True), nullable=False),
    sa.Column("deck_id", postgresql.UUID(as_uuid=True), nullable=False),
    sa.Column("card_id", postgresql.UUID(as_uuid=True), nullable=False),
    sa.Column("position", sa.Integer(), nullable=False),
    sa.ForeignKeyConstraint(["card_id"], ["cards.id"]),
    sa.ForeignKeyConstraint(["deck_id"], ["decks.id"]),
```

```
        sa.PrimaryKeyConstraint("id"),
        sa.UniqueConstraint("deck_id", "position", name="uq_deck_position"),
    )


def downgrade() -> None:
    op.drop_table("deck_cards")
    op.drop_table("decks")
    op.drop_constraint("fk_games_winner_id_players", "games", type_="
        foreignkey")
    op.drop_table("players")
    op.drop_table("cards")
    op.drop_table("images")
    op.drop_table("games")
    op.drop_table("card_types")
    op.drop_table("users")
```

Листинг 4.2 — Заполнение базы данных

```
def cmd_seed(args: argparse.Namespace) -> int:
    from uuid import uuid4

    from sqlalchemy import select

    from application.services.card_factory import generate_cards,
        list_card_types
    from infrastructure.db.models import CardModel, CardTypeModel,
        ImageModel, UserModel

    SessionLocal = _make_session_factory()

    seed_users = [
        ("gandalf", uuid4()),
        ("saruman", uuid4()),
        ("radagast", uuid4()),
    ]

    with SessionLocal() as session:
        for username, user_id in seed_users:
            exists = session.scalar(
                select(UserModel).where(UserModel.username == username)
            )
            if exists:
```

```
      print(f"  skip  {username} (already exists)")
      continue
    session.add(UserModel(id=user_id, username=username))
    print(f"  add   {username}  id={user_id}")

  session.commit()

with SessionLocal() as session:
  existing_card_type_ids = set(session.scalars(select(CardTypeModel.id))
    )
  for card_type in list_card_types():
    if card_type.id in existing_card_type_ids:
      continue
    session.add(
      CardTypeModel(
        id=card_type.id,
        action=card_type.action.value,
        usage_pattern=card_type.usage_pattern.value,
        if_permanent=card_type.if_permanent,
        color=card_type.color,
      )
    )

  session.commit()

with SessionLocal() as session:
  existing_image_ids = set(session.scalars(select(ImageModel.id)))
  existing_titles = set(session.scalars(select(CardModel.title)))
  current_count = len(existing_titles)
  print(f"Current cards in db: {current_count}")
  target_count = args.cards
  if current_count >= target_count:
    print(f"  skip  card seed (already have {current_count} cards)")
  else:
    all_cards = generate_cards(target_count)
    new_cards = [c for c in all_cards if c.title not in existing_titles]
    to_add = target_count - current_count
    new_cards = new_cards[:to_add]
    for card in new_cards:
      if card.image is not None and card.image.id not in
        existing_image_ids:
```

```
            session.add(
              ImageModel(
                id=card.image.id,
                title=card.image.title,
                file=card.image.file,
              )
            )
            existing_image_ids.add(card.image.id)
          session.add(
            CardModel(
              id=card.id,
              title=card.title,
              creature=card.creature,
              card_type_id=card.card_type_id,
              image_id=card.image_id,
              power=card.power,
              echo=card.echo,
              cost=card.cost,
              cool_points=card.cool_points,
            )
          )
        session.commit()
        print(f"  add   {len(new_cards)} cards")

    print("Seed complete.")
    return 0
```

Листинг 4.3 — Создание роли пользователя (гостя)

```
CREATE ROLE cmb_guest NOLOGIN;
GRANT USAGE ON SCHEMA public TO cmb_guest;
GRANT SELECT ON cards, card_types, images TO cmb_guest;
```

Листинг 4.4 — Полный SQL-скрипт: создание ролей и выдача привилегий

```
DO $$
BEGIN
  IF NOT EXISTS (SELECT 1 FROM pg_roles WHERE rolname = 'cmb_guest') THEN
    CREATE ROLE cmb_guest NOLOGIN;
  END IF;
  IF NOT EXISTS (SELECT 1 FROM pg_roles WHERE rolname =
     'cmb_authenticated_user') THEN
    CREATE ROLE cmb_authenticated_user NOLOGIN;
  END IF;
```

```
  IF NOT EXISTS (SELECT 1 FROM pg_roles WHERE rolname = 'cmb_player') THEN
    CREATE ROLE cmb_player NOLOGIN;
  END IF;
  IF NOT EXISTS (SELECT 1 FROM pg_roles WHERE rolname = 'cmb_moderator')
     THEN
    CREATE ROLE cmb_moderator NOLOGIN;
  END IF;
  IF NOT EXISTS (SELECT 1 FROM pg_roles WHERE rolname = 'cmb_master') THEN
    CREATE ROLE cmb_master NOLOGIN;
  END IF;
END
$$;

GRANT USAGE ON SCHEMA public TO cmb_guest, cmb_authenticated_user,
    cmb_player, cmb_moderator, cmb_master;

GRANT SELECT ON cards, card_types, images TO cmb_guest,
    cmb_authenticated_user, cmb_player, cmb_moderator, cmb_master;

GRANT SELECT, INSERT ON games, players TO cmb_authenticated_user,
    cmb_player;
GRANT SELECT, UPDATE ON decks TO cmb_player;
GRANT SELECT, UPDATE, DELETE ON deck_cards TO cmb_player;

GRANT SELECT, INSERT, UPDATE, DELETE ON users, games, players TO
    cmb_moderator, cmb_master;
GRANT SELECT ON decks, deck_cards TO cmb_moderator, cmb_master;

GRANT INSERT, UPDATE, DELETE ON cards, card_types, images TO cmb_master;
```

Листинг 4.5 — Создание роли зарегестрированного пользователя

```
CREATE ROLE cmb_authenticated_user NOLOGIN;
GRANT USAGE ON SCHEMA public TO cmb_authenticated_user;
GRANT SELECT ON cards, card_types, images TO cmb_authenticated_user;
GRANT SELECT, INSERT ON games, players TO cmb_authenticated_user;
```

Листинг 4.6 — Создание роли игрока

```
CREATE ROLE cmb_player NOLOGIN;
GRANT USAGE ON SCHEMA public TO cmb_player;
GRANT SELECT ON cards, card_types, images TO cmb_player;
GRANT SELECT, INSERT ON games, players TO cmb_player;
GRANT SELECT, UPDATE ON decks TO cmb_player;
```

```
GRANT SELECT, UPDATE, DELETE ON deck_cards TO cmb_player;
```

Листинг 4.7 — Создание роли модератора

```
CREATE ROLE cmb_moderator NOLOGIN;
GRANT USAGE ON SCHEMA public TO cmb_moderator;
GRANT SELECT ON cards, card_types, images TO cmb_moderator;
GRANT SELECT, INSERT, UPDATE, DELETE
ON users, games, players TO cmb_moderator;
GRANT SELECT ON decks, deck_cards TO cmb_moderator;
```

Листинг 4.8 — Создание роли мастера

```
CREATE ROLE cmb_master NOLOGIN;
GRANT USAGE ON SCHEMA public TO cmb_master;
GRANT SELECT, INSERT, UPDATE, DELETE
ON users, games, players TO cmb_master;
GRANT SELECT, INSERT, UPDATE, DELETE
ON cards, card_types, images TO cmb_master;
GRANT SELECT ON decks, deck_cards TO cmb_master;
```

Листинг 4.9 — Функция и триггер валидации карты

```
CREATE FUNCTION card_validate_trigger() RETURNS trigger AS $$
DECLARE
    action_name TEXT;
BEGIN
    -- 1. Cost: positive, not greater than 7
    IF NEW.cost <= 0 OR NEW.cost > 7 THEN
        RAISE EXCEPTION 'Card cost must be between 1 and 7, got %', NEW.
            cost;
    END IF;

    -- 2. Power: positive; for buff_* not greater than 2
    IF NEW.power <= 0 THEN
        RAISE EXCEPTION 'Card power must be positive, got %', NEW.power;
    END IF;

    SELECT action INTO action_name
    FROM card_types
    WHERE id = NEW.card_type_id;

    IF action_name LIKE 'buff_%' AND NEW.power > 2 THEN
        RAISE EXCEPTION
```

```
            'Power for buff_* card type must not exceed 2, got %', NEW.
                power;
    END IF;

    -- 3. For buff_* cards creature field is required (location)
    IF action_name LIKE 'buff_%' AND (NEW.creature IS NULL OR NEW.creature
        = '') THEN
        RAISE EXCEPTION
            'buff_* card must have a location in creature field';
    END IF;

    -- 4. Cool points: between -1 and 5
    IF NEW.cool_points < -1 OR NEW.cool_points > 5 THEN
        RAISE EXCEPTION
            'cool_points must be between -1 and 5, got %', NEW.cool_points
                ;
    END IF;

    -- 5. Card type is required
    IF NEW.card_type_id IS NULL THEN
        RAISE EXCEPTION 'Card must have a card type';
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_card_validate
BEFORE INSERT OR UPDATE ON cards
FOR EACH ROW EXECUTE FUNCTION card_validate_trigger();
```

Листинг 4.10 — Хранимая процедура проверки покупки

```
CREATE FUNCTION purchase_card(player_id UUID, card_id UUID) RETURNS VOID
   AS $$
DECLARE
    current_echo INTEGER;
    card_cost INTEGER;
BEGIN
    SELECT cur_echo INTO current_echo FROM players WHERE id = player_id;
    IF current_echo IS NULL THEN
        RAISE EXCEPTION 'Player not found';
    END IF;
```

```
    SELECT cost INTO card_cost FROM cards WHERE id = card_id;
    IF card_cost IS NULL THEN
        RAISE EXCEPTION 'Card not found';
    END IF;

    IF current_echo < card_cost THEN
        RAISE EXCEPTION 'Insufficient echo points for purchase';
    END IF;

    UPDATE players
    SET cur_echo = cur_echo - card_cost
    WHERE id = player_id;
    -- Purchase record may be created separately in a purchase history
       table.
END;
$$ LANGUAGE plpgsql;
```

Приложение Б

Презентация к курсовой работе состоит из 15 слайдов.